

نمبرون للسفر والسياحة · NUMBER ONE TRAVEL & TOURISM

Stage 15 Status Report

Operations Control, Service Management & Business Integration Layer

STATUS: PARTIALLY COMPLETE

Repository: mashhorfoods/mashhor-demo

Branch: claude/awesome-feynman-zyfjar

Commit: f920ec03873a0feb54f4fe9f100c4fb6e1a1c334

Report date: 2026-09-17

0. Executive summary

Stage 15 adds the operations control layer connecting Customer → Supervisor → Booking → Service → Supplier → Payment → Documents → Operations → Notification → Completion. Before writing any code, a full audit of the repository found that **Stage 14 (an admin dashboard) does not exist** — only Stages 10–13 are built. Rather than silently assume it or refuse the stage, this implementation built the minimum authenticated surface Stage 15 itself requires: a shared Admin / Operations Staff login and portal, explicitly *not* a general admin dashboard.

Everything described below has been implemented against the existing Stage 12.2 backend and Stage 13 patterns, is covered by 629 new automated checks (130 backend + 499 browser), and passes the full 21-suite project test gate with zero regressions, zero untranslated strings and zero accessibility findings. Nothing external is connected — no deployed backend, no real supplier, payment or notification-delivery integration — and ten business decisions remain explicitly open (§8).

Key disclosure. Stage 14 was not found to exist in this repository. The Admin/Operations Staff portal built here is scoped strictly to what Stage 15 needs (booking operations, tasks, escalations, service configuration, suppliers, notifications, audit, settings) — it is not a user-management system, a CMS, or a financial reporting suite. A broader Stage 14, if the business wants one, remains unscoped and unbuilt.

1. What was built

1.1 Booking lifecycle state machine

A backend-authoritative, **configurable** state machine (stored in `business_config`, not hard-coded), tracked in a new `ops_status` column on the existing bookings table — kept entirely separate from the customer-facing status the booking engine already owns. The example graph given in the brief was seeded exactly as given and marked `technical_example_pending_business_confirmation`:

```
submitted → pending_review → awaiting_payment → payment_received → processing
           → supplier_pending → confirmed → ticketed → service_in_progress → completed
(+ cancelled / failed / refunded at the appropriate points)
```

- Every transition is rejected (422) unless it is in the current status's allowed list read from config.
- Six statuses are **payment-gated**: entry is rejected (409) unless `payment_status === 'paid'` — the payment provider's own status stays authoritative.
- Every transition writes a `booking_status_history` row (actor, role, reason, metadata, timestamp) and an `audit_events` row, both server-side only.
- The UI offers only the transitions the backend actually allows for the current status and role — an operator cannot even attempt an invalid one.

1.2 Tasks & escalations

A reusable `operation_tasks` system (type, booking, assignee, status, priority, due date, notes) with create/assign/reassign/status change, each gated on the `task.manage` permission and fully audited. A parallel escalations system (reason, severity, assigned team/operator, status Open → Investigating/Waiting → Resolved/Closed). No SLA duration is invented — `business_config.sla_config` is an explicit, empty placeholder for the business to fill in.

1.3 Service management

The existing 13-service registry (flights, hotels, visa, packages, umrah, transport, groups, medical, study, work, issue, change, cancel) gains an operational layer — joined by id, never duplicating name or description — covering active/booking-enabled flags, workflow type, supplier type, ordered **workflow steps**, and per-service **document requirements**. Workflow steps were seeded only for the four services the brief itself gave worked examples for (**flights, visa, hotels, medical**); the other nine are shown honestly as "not yet configured" rather than given an invented generic workflow.

1.4 Document review & supplier operations

Document review (approve/reject with a reason) extends the existing `documents` table from Stage 12.2 rather than creating a second entity. A new `suppliers` directory and a `booking_suppliers` link record a supplier reservation reference and a ticket/confirmation number as two distinct fields (never conflated with the booking's own reference). Every supplier's integration status reads **not_connected** — no real reservation, ticketing or hotel confirmation is made anywhere in this stage, and no supplier credential or secret is ever exposed to the frontend.

1.5 Notes, notifications & audit

Capability	What it does
Customer / internal notes	Hard <code>CHECK(type IN ('customer', 'internal'))</code> constraint; the customer's own account API route only ever reads <code>type='customer'</code> — verified directly by an automated test that an internal note can never reach a customer response.
Notification templates	Event/channel/subject/body in both languages; server-side sanitization strips all HTML and escapes entities before storage, keeping <code>{{variables}}</code> as plain text.
Delivery history	Reuses the existing <code>outbox</code> table from Stage 12.2 rather than a new entity; never claims delivery beyond what that record's own status says.
Audit trail	A new <code>audit_events</code> table, distinct from the scrubbed technical diagnostics table — every actor-attributed action is recorded and visible to staff with <code>audit.view</code> .

1.6 Roles, permissions & the staff portal

A third session tier mirrors the pattern already used for the customer (Stage 12.2) and supervisor (Stage 13) sessions: its own cookie pair (`no_ops_session/no_ops_csrf`), its own session table (`staff_sessions`), fully isolated from the other two. Two roles share it — **Admin** (every permission implicitly) and **Operations Staff** (only what is explicitly granted) — checked against a fixed 14-item permission vocabulary on every mutating route via `requirePermission()`. The frontend's `can()` helper

only ever hides a control the backend would reject anyway; it is documented in code as never the authority, and this was verified directly: an Operations Staff fixture with a named, partial permission set was confirmed — against the real backend, not just the UI — to never see or successfully call a route outside its grant.

2. Verification results

2.1 Full project test gate

Suite	Result	Suite	Result
final	96/96 passed	journey	512 passed, 0 failed
ghx	16/16 passed	account	782 passed, 0 failed
ghm	42/42 passed	integration	518 passed, 0 failed
gfx	62/62 passed	backend	130 passed, 0 failed
home	170/170 passed	supervisor-portal	505 passed, 0 failed
services	146/146 passed	ops-portal	499 passed, 0 failed
detail	470/470 passed	rm	27/27 passed
destinations	165/165 passed	links	0 problems
offers	170/170 passed	i18n	0 untranslated strings
booking	236/236 passed	a11y	0 unique findings
supervisor	182/182 passed		

Result: 21/21 suites green, zero regressions in the pre-existing platform.

2.2 Stage 15 backend tests (tests/backend.mjs, 130 checks)

Staff authentication and role isolation; CSRF enforcement; permission enforcement (server-side, direct API calls — not just hidden UI); state machine (valid transitions, rejected invalid transitions, the payment gate); tasks; escalations; document review; suppliers; note-type isolation; template sanitization; notification history; the audit trail; services and workflow reads.

2.3 Stage 15 browser suite (tests/ops-portal.mjs, 499 checks)

- Authorization: every portal route guarded for a guest; sign-in/out flow
- Populated screens against the development data adapter (dashboard, bookings, detail, tasks, escalations, services, suppliers, notifications, audit, settings)
- Permission gating rendered only through `can()`
- Empty / error / slow states via development switches
- Responsive + RTL/LTR matrix at 390 / 834 / 1440 in Arabic and English

- State-machine transition picker, note isolation (internal text never leaks to the customer notes block)
- **Real-backend pass:** Admin vs. Operations Staff fixture sign-in, permission isolation, note isolation, forbidden-region handling — against the actual Stage 12.2 backend, not a stand-in

2.4 Responsive / RTL / accessibility matrix

Width	Arabic (RTL)	English (LTR)
390 × 844 (mobile)	verified	verified
834 × 1100 (tablet)	verified	verified
1440 × 1000 (desktop)	verified	verified

Each combination checked for: exactly one `<h1>`, no horizontal scroll, no `index` present, no text under 12px, every input/select/textarea accessibly labelled, the portal nav marking the current item, and (English only) zero residual Arabic text.

3. Security posture

Authorization	Server-side <code>requirePermission()</code> on every mutating route; verified by direct API calls bypassing the UI entirely, not just by hiding a button.
Session isolation	Staff / customer / supervisor sessions use separate cookies and tables; a token from one is rejected by every route belonging to another.
CSRF	Double-submit token required on every mutating staff request, same mechanism as the existing customer/supervisor sessions.
CORS / rate limiting	Reused unchanged from the existing Stage 12.2 middleware — no new bypass surface introduced.
Secrets	No supplier credential, API key, or permanent document URL is ever placed in a staff-facing response.
Templates	All HTML stripped and entities escaped server-side before a notification template is stored; no script injection surface.

4. Integration status

Item	Status	Note
State machine, tasks, escalations, service catalogue, document review, supplier directory, notes, templates, audit, permissions	BUILT + VERIFIED	Verified locally against the dev adapter and the real local backend
Deployed backend	NOT CONNECTED	Same gap carried from Stage 12.2 / 13

Real supplier / ticketing integrations	NOT CONNECTED	Every supplier record reads integrationStatus: not_connected
Real payment settlement rules	NOT CONNECTED	Payment provider's own status remains authoritative; no new payment logic added
Outbound notification send pipeline	NOT CONNECTED	Templates and sanitization exist; no SMS/email/push provider is wired up
Real staff identities	FIXTURE ONLY	staff-admin-1 / staff-ops-1 are test accounts

5. Business decisions still required

Per the brief's own instruction, none of the following were decided by this implementation — each is built as configurable architecture with a placeholder marked `pending_business_confirmation` in `business_config`:

1. The final booking lifecycle graph (§1.1's graph is a worked example, not a confirmed rule)
2. SLA durations for escalations
3. Task priority levels and their meaning
4. Document requirements per service (none seeded)
5. Workflow steps for the nine services without a seeded example
6. Real supplier integrations
7. The notification send pipeline (provider, delivery guarantees)
8. Commission, refund and cancellation policy (carried over unresolved from Stage 13)
9. Real staff provisioning process
10. Scope of a genuine Stage 14 admin dashboard, if wanted beyond what Stage 15 needed

6. Files changed

Path	Contents
backend/migrations/003_operations.sql	staff, staff_sessions, ops_status/assigned_operator, booking_status_history, operation_tasks, escalations, services, service_workflows, service_document_requirements, documents.review_*, suppliers, booking_suppliers, booking_notes, notification_templates, audit_events, business_config seeds
backend/staff.mjs · staff-routes.mjs	Identity, permissions, state machine, tasks/escalations, services, suppliers, notes, templates, audit — every route handler

backend/{http,server,db,routes,fixtures}.mjs	Staff cookies, gated routing, migration/seed wiring, customer-note scoping, fixtures
assets/js/ops/*	auth.js, data.js, 6 adapters, 10 UI screens (dashboard, bookings, tasks, escalations, services, suppliers, notifications, audit, settings, auth-screens)
assets/css/24-ops-portal.css	Inline action forms, workflow-step list, audit-trail columns (reuses Stage 13's 23-supervisor-portal.css for everything else)
admin/*/index.html	13 pages: dashboard, bookings, tasks, escalations, services, suppliers, notifications, audit, settings, sign-in, forgot-password, reset-password, sign-out
assets/js/core/strings/{ar,en}.js	ops.* and page.ops.* strings, full key parity verified (1490 keys per language)
tests/backend.mjs · tests/ops-portal.mjs	130 + 499 new automated checks
docs/STAGE-15-OPERATIONS-CONTROL.md	Full architecture, API contracts, security notes, business-decision register

7. Recommendation

Resolve the ten business decisions in §5, particularly the final lifecycle graph and per-service document requirements, since most other stages' scope depends on them. After that, the next stage should be either a genuinely scoped Stage 14 (if the business wants a broader admin dashboard than Stage 15 built) or a real integration stage wiring one supplier, one payment settlement path, and one notification channel end to end. **No next stage has been started automatically.**